Open in Colab

ML Homework 4 Guide

1. Save a copy of this ipynb file in your GoogleDrive or PC.
2. Edit the name of this code from "HW4.ipynb" to "HW4_(your name).ipynb"
3. Fill out the code cells below according to descriptions.
4. Save and upload to BrightSpace. (DO NOT clear the outputs of your code)
5. Convert the .ipynb file to PDF file and upload together
6. Upload the saved confusion matrix image file together

[HW 4] Grid search for CNN

1. Load the 'AccData.csv' file as you loaded in the 'ML7_Code2'.
2. Prepare training and test dataset with the test data ratio of """30%""".
3. Perform a grid search to find the best combination of hyperparameters for the CNN model.

- You can create any combination of hyperparameters.
- Do not copy 'ML7_Code2' exactly. Set at least one other hyperparamter as the grid search target.

4. Determine your best CNN model based on classification accuracy.
5. Plot a confusion matrix for the best model and save it as an image file (.png or .jpg).

- Search how to save a figure as an image file.
- Upload the confusion matrix image to BrightSpace with ipynb and pdf files

Import Packages

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Import 'Tensorflow' pakage
import tensorflow as tf
from tensorflow import keras
from scipy import signal
from sklearn.model_selection import train_test_split
```

In [2]:
```
# Access to google drive
from google.colab import drive

drive.mount("/content/drive")
```

```
Mounted at /content/drive
```

Load the 'AccData.csv' File from ML7_Code2

In [3]:
```
AccData_pd_load = pd.read_csv(
    "https://github.com/purduelamm/purdue_me597_iiot/blob/main/ml_tutorial/Dataset_Acc/AccData.csv?raw=true"
).iloc[:, 1:]
AccData_pd_load.shape
AccData = np.array(AccData_pd_load)
AccData.shape
```

Out[3]:
```
(360, 2774)
```

Prepare Training and Test Dataset with the test data ratio of 30%

In [4]:
```
# Convert to Spectogram by STFT
Fs = 12800  # Sampling Frequency
f, t, AccSTFT = signal.spectrogram(AccData, Fs, nperseg=78, noverlap=10)
AccSTFT.shape

# Split Training & Test Data
NoOfData = 180

NormalSet = AccSTFT[:NoOfData]
AbnormalSet = AccSTFT[NoOfData:]

NoOfSensor = 1
NormalSet = NormalSet.reshape(
    NormalSet.shape[0], NormalSet.shape[1], NormalSet.shape[2], NoOfSensor
)
AbnormalSet = AbnormalSet.reshape(
    AbnormalSet.shape[0],
    AbnormalSet.shape[1],
    AbnormalSet.shape[2],
    NoOfSensor,
)

NormalSet.shape, AbnormalSet.shape

# Designate test data ratio
TestData_Ratio = 0.3  # THIS IS WHERE THE RATIO IS SET TO 30%

TrainData_Nor, TestData_Nor = train_test_split(
    NormalSet, test_size=TestData_Ratio, random_state=777
)
TrainData_Abn, TestData_Abn = train_test_split(
    AbnormalSet, test_size=TestData_Ratio, random_state=777
)

print(TrainData_Nor.shape, TestData_Nor.shape)
print(TrainData_Abn.shape, TestData_Abn.shape)
```

```
(126, 40, 40, 1) (54, 40, 40, 1)
(126, 40, 40, 1) (54, 40, 40, 1)
```

Data Labeling and Preparation

In [5]:
```
TrainLabel_Nor = np.zeros((TrainData_Nor.shape[0], 2))
TrainLabel_Abn = np.ones((TrainData_Abn.shape[0], 2))
TestLabel_Nor = np.zeros((TestData_Nor.shape[0], 2))
TestLabel_Abn = np.ones((TestData_Abn.shape[0], 2))

TrainLabel_Nor[:, 0] = 1  # [1,0]: Normal
TrainLabel_Abn[:, 0] = 0  # [0,1]: Abnormal
TestLabel_Nor[:, 0] = 1  # [1,0]: Normal
TestLabel_Abn[:, 0] = 0  # [0,1]: Abnormal

print(TrainLabel_Nor.shape, TestLabel_Nor.shape)
print(TrainLabel_Abn.shape, TestLabel_Abn.shape)

TrainData = np.concatenate([TrainData_Nor, TrainData_Abn], axis=0)
TestData = np.concatenate([TestData_Nor, TestData_Abn], axis=0)
TrainLabel = np.concatenate([TrainLabel_Nor, TrainLabel_Abn], axis=0)
TestLabel = np.concatenate([TestLabel_Nor, TestLabel_Abn], axis=0)

print(TrainData.shape, TestData.shape)
print(TrainLabel.shape, TestLabel.shape)
```

```
(126, 2) (54, 2)
(126, 2) (54, 2)
(252, 40, 40, 1) (108, 40, 40, 1)
(252, 2) (108, 2)
```

Hyperparameters with at least one different target from ML7_Code2 (Learning Rate)

In [11]:
```python
# Hyperparameters for grid search
param_FiltS = [3, 5]  # filter(kernel) size (only convolution layer)
param_FiltN = [2, 4]  # number of filters   (only convolution layer)
param_Strid = [1, 2]  # stride              (only convolution layer)
learningRate = [1e-3, 1e-4]

# Fixed hyperparameters
noOfNeuron = 10
Epoch = 1000

# Calculate the number of cases
NoOfCases = len(param_FiltS) * len(param_FiltN) * len(param_Strid)
NoOfCases
```

Out[11]: 8

CNN Model Definition

In [7]:
```python
def CNN_model(
    input_data, noOfNeuron, learningRate, filterSize, numOfFilters, stride
):
    model = keras.Sequential()
    model.add(keras.layers.Input(shape=input_data.shape[1:]))
    model.add(
        keras.layers.Conv2D(
            filters=numOfFilters,
            kernel_size=(filterSize, filterSize),
            strides=(stride, stride),
            activation="relu",
            padding="same",
        )
    )
    model.add(keras.layers.MaxPooling2D(pool_size=(2, 2)))
    model.add(
        keras.layers.Conv2D(
            filters=numOfFilters * 2,
            kernel_size=(filterSize, filterSize),
            strides=(1, 1),
            activation="relu",
            padding="same",
        )
    )
    model.add(keras.layers.MaxPooling2D(pool_size=(2, 2)))
    model.add(keras.layers.Flatten())
    model.add(keras.layers.Dense(noOfNeuron, activation="relu"))
    model.add(keras.layers.Dense(2, activation="softmax"))

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learningRate),
        loss="categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model
```

In [17]:
```python
# Create an empty dataframe to store the accuracy results
Accuracy_df = pd.DataFrame(
    np.zeros(shape=(NoOfCases, 5)),
    columns=["filter size", "number of filters", "stride", "learning rate", "Accuracy"],
```

```
)
Accuracy_df
```

Out[17]:

	filter size	number of filters	stride	learning rate	Accuracy
0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0
5	0.0	0.0	0.0	0.0	0.0
6	0.0	0.0	0.0	0.0	0.0
7	0.0	0.0	0.0	0.0	0.0

Training with different HyperParameters

In [19]:

```
import itertools

# Initialize a count value to store the performance of each model
cnt = 0

# Iterate through all combinations of hyperparameters
for filtS, filtN, strid, lr in itertools.product(
    param_FiltS, param_FiltN, param_Strid, learningRate
):
    # Create and train the model
    model = CNN_model(TrainData, noOfNeuron, lr, filtS, filtN, strid)
    model.fit(TrainData, TrainLabel, epochs=Epoch, verbose=0)

    # Evaluate on test data
    test_loss, test_accuracy = model.evaluate(TestData, TestLabel, verbose=0)

    # Store results in dataframe
    Accuracy_df.loc[cnt] = [filtS, filtN, strid, lr, test_accuracy]

    print(f"Model: {cnt}")
    print(f"Filter size   : [{filtS},{filtS}]")
    print(f"Num of Filters: {filtN}")
    print(f"Strides       : {strid}")
    print(f"Learning Rate : {lr}")
    print(f"Accuracy      : {test_accuracy}")

    cnt += 1

# Display the dataframe
Accuracy_df
```

```
Model: 0
Filter size   : [3,3]
Num of Filters: 2
Strides       : 1
Learning Rate : 0.001
Accuracy      : 0.9814814925193787
Model: 1
Filter size   : [3,3]
Num of Filters: 2
Strides       : 1
Learning Rate : 0.0001
Accuracy      : 0.5925925970077515
Model: 2
Filter size   : [3,3]
Num of Filters: 2
Strides       : 2
Learning Rate : 0.001
Accuracy      : 0.9722222089767456
Model: 3
Filter size   : [3,3]
Num of Filters: 2
Strides       : 2
Learning Rate : 0.0001
Accuracy      : 0.9722222089767456
Model: 4
Filter size   : [3,3]
Num of Filters: 4
Strides       : 1
Learning Rate : 0.001
Accuracy      : 0.9814814925193787
Model: 5
Filter size   : [3,3]
Num of Filters: 4
Strides       : 1
Learning Rate : 0.0001
Accuracy      : 0.9537037014961243
Model: 6
Filter size   : [3,3]
Num of Filters: 4
Strides       : 2
Learning Rate : 0.001
Accuracy      : 0.9722222089767456
Model: 7
Filter size   : [3,3]
Num of Filters: 4
Strides       : 2
Learning Rate : 0.0001
Accuracy      : 0.9444444179534912
Model: 8
Filter size   : [5,5]
Num of Filters: 2
Strides       : 1
Learning Rate : 0.001
Accuracy      : 0.9814814925193787
Model: 9
Filter size   : [5,5]
Num of Filters: 2
Strides       : 1
Learning Rate : 0.0001
Accuracy      : 0.9629629850387573
Model: 10
Filter size   : [5,5]
Num of Filters: 2
Strides       : 2
Learning Rate : 0.001
Accuracy      : 0.9722222089767456
Model: 11
Filter size   : [5,5]
Num of Filters: 2
Strides       : 2
```

```
Learning Rate : 0.0001
Accuracy      : 0.7592592835426331
Model: 12
Filter size   : [5,5]
Num of Filters: 4
Strides       : 1
Learning Rate : 0.001
Accuracy      : 0.9722222089767456
Model: 13
Filter size   : [5,5]
Num of Filters: 4
Strides       : 1
Learning Rate : 0.0001
Accuracy      : 0.9722222089767456
Model: 14
Filter size   : [5,5]
Num of Filters: 4
Strides       : 2
Learning Rate : 0.001
Accuracy      : 0.9722222089767456
Model: 15
Filter size   : [5,5]
Num of Filters: 4
Strides       : 2
Learning Rate : 0.0001
Accuracy      : 0.9722222089767456
```

Out[19]:

	filter size	number of filters	stride	learning rate	Accuracy
0	3.0	2.0	1.0	0.0010	0.981481
1	3.0	2.0	1.0	0.0001	0.592593
2	3.0	2.0	2.0	0.0010	0.972222
3	3.0	2.0	2.0	0.0001	0.972222
4	3.0	4.0	1.0	0.0010	0.981481
5	3.0	4.0	1.0	0.0001	0.953704
6	3.0	4.0	2.0	0.0010	0.972222
7	3.0	4.0	2.0	0.0001	0.944444
8	5.0	2.0	1.0	0.0010	0.981481
9	5.0	2.0	1.0	0.0001	0.962963
10	5.0	2.0	2.0	0.0010	0.972222
11	5.0	2.0	2.0	0.0001	0.759259
12	5.0	4.0	1.0	0.0010	0.972222
13	5.0	4.0	1.0	0.0001	0.972222
14	5.0	4.0	2.0	0.0010	0.972222
15	5.0	4.0	2.0	0.0001	0.972222

Determine best CNN model based on classification accuracy

In [20]:

```
# Sort the Accuracy_df by 'Accuracy' column in descending order
Accuracy_df_sorted = Accuracy_df.sort_values(by='Accuracy', ascending=False).reset_index(drop=True)

# Output the best case
Best_FiltS = int(Accuracy_df_sorted.iloc[0, 0])
Best_FiltN = int(Accuracy_df_sorted.iloc[0, 1])
Best_Strid = int(Accuracy_df_sorted.iloc[0, 2])
Best_learningRate = Accuracy_df_sorted.iloc[0, 3]
```

```
print(f"[Best case]\n" +
      f"Filter size   : [{Best_FiltS},{Best_FiltS}]\n" +
      f"Num of Filters: {Best_FiltN}\n" +
      f"Strides       : {Best_Strid}\n" +
      f"Learning Rate : {Best_learningRate}\n" +
      "Accuracy: %.2f" % (Accuracy_df_sorted.iloc[0, 3]))

# Calculate mean and standard deviation accuracy for each filter size
mean_accuracy_FiltS = Accuracy_df.groupby(['filter size'])['Accuracy'].agg(['mean', 'std']).reset_index()
mean_accuracy_FiltS

# Calculate mean and standard deviation of accuracy for each number of filter
mean_accuracy_FiltN = Accuracy_df.groupby(['number of filters'])['Accuracy'].agg(['mean', 'std']).reset_inde
mean_accuracy_FiltN

# Calculate mean and standard deviation of accuracy for each stride
mean_accuracy_Strid = Accuracy_df.groupby(['stride'])['Accuracy'].agg(['mean', 'std']).reset_index()
mean_accuracy_Strid

# Calculate mean and standard deviation of accuracy for each learning rate
mean_accuracy_lr = Accuracy_df.groupby(['learning rate'])['Accuracy'].agg(['mean', 'std']).reset_index()
mean_accuracy_lr
```

```
[Best case]
Filter size   : [3,3]
Num of Filters: 2
Strides       : 1
Learning Rate : 0.001
Accuracy: 0.00
```

Out[20]:

	learning rate	mean	std
0	0.0001	0.891204	0.140489
1	0.0010	0.975694	0.004792

In [24]:
```
# Re-create and train the best model using the CNN_model function
# TrainData, noOfNeuron, Epoch are available in the kernel state.
best_cnn_model = CNN_model(TrainData, noOfNeuron, Best_learningRate, Best_FiltS, Best_FiltN, Best_Strid)
best_cnn_model.fit(TrainData, TrainLabel, epochs=Epoch, verbose=0) # verbose=0 to avoid too much output
```

Out[24]:
```
<keras.src.callbacks.history.History at 0x7b8da406a330>
```

In [26]:
```
import os

# Define the directory to save the model
save_directory = '/content/drive/MyDrive/Colab Notebooks/SavedFiles/ME 597 IIoT/ML_Models/GridSearch_CNN/'
if not os.path.exists(save_directory):
    os.mkdir(save_directory)

# Model Name
best_cnn_model_name = f'BEST_CNN_FS{Best_FiltS}_FN{Best_FiltN}_St{Best_Strid}_Lr{Best_learningRate}'

# Save the model
best_cnn_model.export(save_directory + best_cnn_model_name)

print(f"The best CNN model (Filter Size: {Best_FiltS}, Number of Filters: {Best_FiltN}, Stride: {Best_Strid}
```

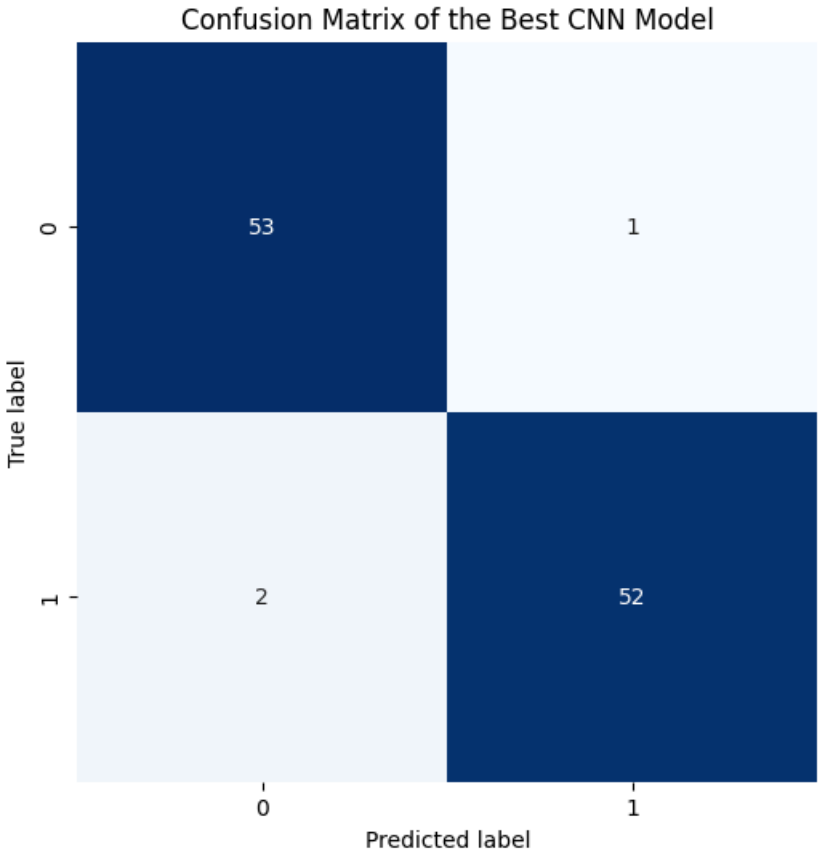
```
Saved artifact at '/content/drive/MyDrive/Colab Notebooks/SavedFiles/ME 597 IIoT/ML_Models/GridSearch_CNN/BEST_CNN_FS3_FN2_St1_Lr0.001'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 40, 40, 1), dtype=tf.float32, name='keras_tensor_1334')
Output Type:
  TensorSpec(shape=(None, 2), dtype=tf.float32, name=None)
Captures:
  135849080803344: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135848511403408: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135849080803728: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135848273210128: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135848273210704: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135848273208784: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135848273211088: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135848273210896: TensorSpec(shape=(), dtype=tf.resource, name=None)
The best CNN model (Filter Size: 3, Number of Filters: 2, Stride: 1, Learning Rate: 0.001) has been saved to: /content/drive/MyDrive/Colab Notebooks/SavedFiles/ME 597 IIoT/ML_Models/GridSearch_CNN/BEST_CNN_FS3_FN2_St1_Lr0.001
```

Confusion Matrix

In [29]:

```python
# Predict the output (Robotic spot-welding condition) for the test data
Predicted = best_cnn_model.predict(TestData)

# Convert TestLabel and Predicted into vectors to calculate the confusion matrix and evaluation metrics
TestLabel_rev = np.argmax(TestLabel, axis=1)
Predicted_rev = np.argmax(Predicted, axis=1)

# Plot the confusion matrix
import seaborn as sns
from sklearn.metrics import confusion_matrix

# Calculate the confusion matrix
cm = confusion_matrix(TestLabel_rev, Predicted_rev)

plt.figure(figsize=(6, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap=plt.cm.Blues, cbar=False, square=True)
plt.xlabel("Predicted label")
plt.ylabel("True label")
plt.title("Confusion Matrix of the Best CNN Model")
plt.show()
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 2s 339ms/step
```



Evaluation Metrics

In [30]:
```
from sklearn import metrics

# Calculate the evaluation metrics
accuracy  = metrics.accuracy_score(TestLabel_rev, Predicted_rev)
precision = metrics.precision_score(TestLabel_rev, Predicted_rev)
recall    = metrics.recall_score(TestLabel_rev, Predicted_rev)
f1_score  = metrics.f1_score(TestLabel_rev, Predicted_rev)

# Print the evaluation metrics
print(f"Best CNN Model Evaluation:\n")
print(f"Accuracy : {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall   : {recall:.2f}")
print(f"F1 Score : {f1_score:.2f}")
```

```
Best CNN Model Evaluation:

Accuracy : 0.97
Precision: 0.98
Recall   : 0.96
F1 Score : 0.97
```

ML7 and ML8 Summary and Deliverables

Answer the following questions for your achievements

Q1. Please summarize ML7 and ML8.

In ML7, we continued learning more about artificial and convolutional neural networks and learned how to perform a grid search using different hyperparameters to find the best model. We continued to learn how to prepare a data processing

pipeline using a FeatureData dataset for the ANN, and an acceleration dataset for the CNN. Finally, we learned how to verify which model was best after the outcome of the grid search.

In ML8, we analyzed acceleration data once again but this time applied it to fault detection. First, we preprocessed the vibration data to a different frequency and normalized it to a -1.0 to 1.0 scale, since the YAMNet model we are using expects this normalization. We then assigned labels like we do with other ML models. Finally, we trained a model and evaluated it similarly to previous ML examples.

Q2. What skills did you have to develop to accomplish this project?

To accomplish this project, I needed to learn more about how to train ANN and CNN models and how to properly perform a grid search on different hyperparameters.

Q3. What aspects of this project were the most beneficial for your learning?

The aspects of the project that were most beneficial is how these relate to the labs that we are doing using vibration data.

Q4. What challenges did you encounter in completing the project?

The challenges I encountered with this project were that training CNN models is very computationally expensive and takes a long time.

Q5. How did you overcome the challenges or remedy the problems encountered?

To overcome this issue, I used Google's servers on Colab and used their T4 GPU to speed up model training.
